Curso Avançado

Capítulo 2 - Controle simultâneo de várias composições

Coordenar mais de um trem sem permitir que duas composições utilizem recursos incompatíveis.

Introdução	No capítulo anterior, uma rota era acompanhada isoladamente. Agora, cada composição terá seu próprio estado, enquanto um coordenador único controla as reservas dos trechos compartilhados.
Objetivo	Permitir operações independentes quando os percursos não entram em conflito e impedir autorizações simultâneas para recursos comuns.
Meta de aprendizagem	Distinguir o estado de cada trem da disponibilidade dos recursos e testar reservas de rotas com duas composições.

1. Simultâneo não significa independente

Dois trens podem circular ao mesmo tempo em percursos compatíveis. Quando ambos precisam de um mesmo trecho, desvio ou região de cruzamento, o sistema deve decidir quem passa primeiro e manter o outro aguardando em um local adequado.

Um comando de velocidade pertence a uma composição ou circuito de tração. Já a ocupação e a reserva pertencem aos recursos da ferrovia. Misturar essas informações pode liberar um trecho ainda necessário a outro trem.

Neste capítulo, o controle é simulado no Monitor Serial. Duas composições disputam uma região comum. Nenhuma mensagem aciona a tração, e os nomes de recursos não representam as rotas oficiais da EFMN 74.

2. Três informações que precisam permanecer separadas

Informação	O que representa
Estado da composição	A etapa atual daquele trem: aguardando, reservado, autorizado ou aguardando liberação.
Reserva do recurso	Qual composição tem permissão exclusiva para utilizar o recurso.
Ocupação física	Se o trem ou parte dele está no recurso. Exige informação confiável, não apenas uma variável de reserva.

Um recurso pode estar reservado antes de o trem entrar. Também pode continuar ocupado após a locomotiva sair. Por isso, uma reserva não substitui a detecção física e um evento de saída não libera automaticamente o último vagão.

3. Percursos do exercício

Composição	Recursos necessários
Trem 1	WEST + SHARED: acesso oeste e região comum.
Trem 2	EAST + SHARED: acesso leste e a mesma região comum.

WEST e EAST são acessos exclusivos no exemplo. SHARED é utilizado pelas duas rotas. Assim, estas duas rotas não podem receber autorização simultânea. Enquanto uma composição mantém SHARED reservado, a solicitação da outra permanece recusada.

Para permitir dois movimentos simultâneos, seria necessário cadastrar percursos sem recursos conflitantes e verificar a capacidade da tração de comandá-los separadamente. O exercício se concentra na decisão sobre reservas.

4. Reserva integral da rota

Primeiro, confira todos os recursos. Somente quando todos estiverem disponíveis, registre a reserva de todos eles. Se um estiver indisponível, não reserve nenhum. Isso evita que cada trem guarde uma parte do percurso e fique esperando o recurso reservado pelo outro.

No exemplo, um único loop toma essas decisões sequencialmente. Em um sistema com vários controladores, um coordenador deve arbitrar as reservas. Duas gavetas não devem conceder separadamente permissão para a mesma região.

5. Autorização e liberação por composição

Depois de reservar, ainda é necessário preparar os desvios e conferir suas posições. A autorização pertence à composição selecionada. O sensor detectado deve ser associado à rota e à etapa esperada, sem presumir que o Hall identifica a locomotiva.

O código usa confirmação manual do percurso e da saída completa. Um comando de saída coloca o trem em espera de liberação. Só a conferência de todos os recursos permite retirar sua reserva.

6. Comandos para o Monitor Serial

Comando	Significado
I	Confirmar parada de todas as composições e todos os recursos livres. Inicializa ou recupera o sistema.
1 / 2	Selecionar a composição que receberá os próximos comandos.
P	Solicitar a reserva integral do percurso selecionado.
K	Confirmar desvios e demais condições do percurso.
G	Autorizar circulação simulada da composição selecionada.
A / B	Simular entrada / passagem pelo ponto de saída.
L	Confirmar parada e saída completa, liberando os recursos do trem selecionado.
X	Solicitar emergência global. Mantém as reservas.

Use letras maiúsculas. Os eventos A e B já representam eventos reconhecidos e associados ao trem selecionado. A associação automática dos sensores aos percursos e a identificação das locomotivas serão aprofundadas nos próximos capítulos.

7. Falhas e prioridade de passagem

Uma solicitação P recusada por conflito mantém o trem aguardando. Ela não interrompe a operação válida da outra composição. Um evento fora de sequência, uma emergência ou um prazo excedido bloqueia o sistema e solicita a parada de todos os trens.

A primeira solicitação válida obtém a reserva. Este exemplo não possui fila automática nem prioridade fixa. Após a liberação, o operador solicita novamente a rota que estava aguardando.

8. Código em C++ - coordenador de recursos

Reúna as duas partes em um único sketch. Configure o Monitor Serial em 115200 baud. Os nomes e comentários do código estão em inglês. Não conecte este exemplo a saídas de tração.

```
enum class TrainState { WAITING, RESERVED, READY,
                        RUNNING, OCCUPIED, WAIT_CLEAR };
struct Train {
  TrainState state;
  unsigned long changedAt;
};
Train trains[2] = {
  {TrainState::WAITING, 0}, {TrainState::WAITING, 0}
};
// Resource indices: WEST = 0, EAST = 1, SHARED = 2.
const int paths[2][2] = {{0, 2}, {1, 2}};
int owners[3] = {-1, -1, -1}; // -1 means unreserved.
int selected = 0;
bool blocked = true;
const unsigned long timeoutMs = 60000UL;

void setState(int id, TrainState next) {
  trains[id].state = next;
  trains[id].changedAt = millis();
}
void stopAll(const char* reason) {
  blocked = true;
  Serial.println("SIMULATION: stop request for ALL trains.");
  Serial.println(reason);
  // Keep every existing reservation until inspection.
}
bool reservePath(int id) {
  for (int r : paths[id]) {
    if (owners[r] != -1) return false;
  }
  for (int r : paths[id]) owners[r] = id;
  return true;
}
void releasePath(int id) {
  for (int r : paths[id]) {
    if (owners[r] == id) owners[r] = -1;
  }
}
void handleCommand(char c) {
  if (c == 'X') { stopAll("Emergency"); return; }
  if (c == '1' || c == '2') {
    selected = c - '1';
    Serial.print("Selected train: ");
    Serial.println(selected + 1);
    return;
  }
  if (c == 'I' && blocked) {
    // Operator confirms ALL trains stopped and resources clear.
    for (int &owner : owners) owner = -1;
    for (int i = 0; i < 2; ++i)
      setState(i, TrainState::WAITING);
    blocked = false;
    Serial.println("Inspection accepted. System available.");
    return;
  }
```

9. Código em C++ - decisões de cada composição

```
  if (blocked) {
    Serial.println("Blocked. Inspect before sending I.");
    return;
  }
  TrainState st = trains[selected].state;
  if (c == 'P' && st == TrainState::WAITING) {
    if (!reservePath(selected)) {
      Serial.println("Conflict. Train must remain waiting.");
      return;
    }
    setState(selected, TrainState::RESERVED);
    Serial.println("Path reserved. Prepare and check turnouts.");
  } else if (c == 'K' && st == TrainState::RESERVED) {
    setState(selected, TrainState::READY);
    Serial.println("Path checked. Await authorization.");
  } else if (c == 'G' && st == TrainState::READY) {
    setState(selected, TrainState::RUNNING);
    Serial.println("SIMULATION: selected train authorized.");
  } else if (c == 'A' && st == TrainState::RUNNING) {
    setState(selected, TrainState::OCCUPIED);
    Serial.println("Entry event accepted. Reservations kept.");
  } else if (c == 'B' && st == TrainState::OCCUPIED) {
    setState(selected, TrainState::WAIT_CLEAR);
    Serial.println("SIMULATION: selected train stop request.");
  } else if (c == 'L' && st == TrainState::WAIT_CLEAR) {
    // Operator confirms complete exit and stopped train.
    releasePath(selected);
    setState(selected, TrainState::WAITING);
    Serial.println("Selected train resources released.");
  } else {
    stopAll("Unexpected command or sensor event");
  }
}
void setup() {
  Serial.begin(115200);
  stopAll("Startup: inspect all resources before I.");
}
void loop() {
  if (!blocked) {
    for (int i = 0; i < 2; ++i) {
      if (trains[i].state != TrainState::WAITING &&
          millis() - trains[i].changedAt >= timeoutMs) {
        stopAll("Stage timeout");
        break;
      }
    }
  }
  while (Serial.available() > 0) {
    char c = Serial.read();
    if (c != '\n' && c != '\r' && c != ' ') handleCommand(c);
  }
}
```

O prazo de 60 segundos é didático e renova-se a cada mudança de estado. Em operação real, cada etapa terá limites validados. Uma mensagem de parada não comprova parada física.

10. Roteiro de testes com duas composições

Sequência	Resultado esperado
I 1 P K G A	Trem 1 ocupa a rota simulada e mantém WEST e SHARED reservados.
2 P	Conflito. Trem 2 permanece aguardando e não reserva EAST.
1 B	Pedido de parada do trem 1. SHARED continua reservado.
2 P	Novo conflito, pois ainda não houve liberação completa.
1 L 2 P K G A B L	Após a conferência do trem 1, trem 2 consegue reservar e completar o ciclo.
I 1 P K G B	Evento de saída fora de ordem. Bloqueio global.
X durante qualquer movimento	Pedido de parada de todos os trens e reservas preservadas.
Reiniciar o controlador	Sistema bloqueado. Exige nova inspeção, sem retomada automática.

Para repetir um teste após falha, envie I somente depois de conferir todas as composições paradas e todos os recursos livres. Se algum trem permanecer no percurso, mantenha o bloqueio. A inspeção global não substitui a liberação individual durante uma operação normal.

11. Tração analógica e DCC

Em tração analógica, locomotivas alimentadas pelo mesmo circuito recebem a mesma tensão e polaridade. O programa não consegue comandá-las separadamente apenas atribuindo números aos trens. É necessário planejar circuitos isolados e suas interfaces de controle.

Em DCC, comandos podem ser destinados a endereços de locomotivas. Isso permite controle individual, mas não evita conflitos de percurso. As reservas, a confirmação dos desvios e a ocupação continuam necessárias.

12. Exercício e registro no Diário de Bordo

Desenhe duas rotas reais e marque os recursos exclusivos e compartilhados. Inclua qualquer desvio ou região onde as composições possam se atingir, mesmo que os trechos tenham nomes diferentes. Identifique o ponto onde cada trem deve aguardar.

Registre a ordem das solicitações, as reservas concedidas, os conflitos e a liberação. Depois, desenhe dois percursos compatíveis e explique quais condições permitiriam sua utilização simultânea. Confira a tração e a ocupação antes de alterar a montagem.